



Képfeldolgozás
(GKNB_INTM152) 2025-26/2

Kártya számláló alkalmazás

Molnár Ádám
NK: TLXEEE

Tartalom

1.	Bevezetés	4
2.	Elméleti háttér ismertetése	5
2.1.	Színterek konverziója és a digitális képreprezentáció	5
2.2.	Térbeli szűrés és Gaussi zajcsökkentés	5
2.3.	Íránymenti gradiensek és éldetektálás	6
2.3.1.	Gradiens irányának és nagyságának meghatározása.....	6
2.3.2.	Nem-maximális elnyomás	6
2.3.3.	Hiszterézises küszöbölés.....	6
2.4.	Matematikai morfológia alkalmazása.....	6
2.5.	Kontúrkeresés és poligon-approximáció.....	7
2.6.	Geometriai projektivitás és a Homográfia leképezés.....	7
2.7.	Binarizálás a lokális területeken (Otsu-módszere)	7
2.8.	Sablonillesztés és pontozó algoritmusok	8
2.8.1.	Template matching.....	8
2.8.2.	IoU metrika (Intersection over Union)	8
2.8.3.	Kontúr és alakzat elemző metrika (Match Shape).....	8
3.	A megvalósítás terve és kivitelezése	9
3.1.	Felhasznált technológiák és a rendszer architektúrája	9
3.1.1.	OpenCV (cv2).....	9
3.1.2.	NumPy	9
3.1.3.	Matplotlib.....	9
3.1.4.	Pandas.....	9
3.2.	A projekt könyvtárstruktúrája és globális paraméterei	10
3.3.	Kártyalap izolálása és perspektivikus korrekciója	10
3.3.1.	Kép beolvasása és szürkeárnyaltos konverzió.....	10
3.3.2.	Kép skálázása és zajszűrés	10
3.3.3.	Kontúrok keresése.....	10
3.3.4.	Tartalék vonalvezetés	11
3.3.5.	Sarkok rendezése és síkba fordítás	11
3.4.	Dinamikus szimbólumkivágás az index sarokból	11
3.5.	Szimbólumok binarizálása és Normalizálása	12

3.6.	A sablonillesztés megvalósítása és összevont kiértékelése	12
3.7.	Összekapcsolás és a rendszer végrehajtási (fő) folyamata	12
3.8.	Tömeges kiértékelés és tesztelés	13
3.8.1.	A teszthalmaz és a bemenetek kezelése	13
3.8.2.	A tesztelési iteráció	13
3.8.3.	Eredmények naplózása és összesítés	14
4.	Tesztelés	15
4.1.	Egy kártya tesztelése	15
4.2.	Összes kártya tesztelése	16
4.3.	Több kártya a képen	17
4.4.	Jellemző hibaforrások és tévesztések	17
4.4.1.	Suit-ok hasonlósága	17
4.4.2.	Értékek felcserélése	18
4.4.3.	Kártya elcsúszása	18
4.4.4.	Eltérő háttér	18
5.	Felhasználói leírás	19
5.1.	Projekt könyvtárszerkezete	19
5.2.	Egyetlen kártya felismerése	19
5.3.	Tömeges tesztelés és kiértékelés	20
6.	Irodalomjegyzék	21

1. Bevezetés

A jelen dokumentációban bemutatott projekt célja egy olyan képfeldolgozó alkalmazás tervezése és megvalósítása, amely képes digitális képeken szereplő hagyományos francia kártyalapok automatikus detektálására és azonosítására. Az implementált rendszer elvárása, hogy egy kamerakép alapján megtalálja a kártyák térbeli elhelyezkedését megkeresve a lapok kontúráját és sarokpontjait, majd digitálisan vágja és szabványméretre hozza azokat. Ezek után az algoritmusnak a kivágott szegmenseken fel kell ismernie a lapok értékét (rangját: 2-10, J, Q, K, A) és színét (suit: kőr, káró, treff, pikk). A megvalósítás Python programozási nyelven történik, nyílt forráskódú gépi látás könyvtárak, elsősorban az OpenCV felhasználásával.

Az alkalmazás célja, hogy a lehető legnagyobb pontossággal tudja a kártyalapokat felismerni. Ezt megnehezítik a képek közötti apróbb eltérések, mint például kameraszög, fényviszonyok, esetleges becsillanások és hogy a kamerának éppen mennyire sikerült fókuszálnia az adott kártyalapra.

Az alkalmazás teszteléséhez saját adatbázist hoztam létre. Ehhez saját telefonnal fényképeztem be két pakli franciakártyát, tehát 104 lapot. Ez a pakli annyiban eltér egy átlagos francia kártyapaklitól, hogy a figurás lapokon a lap nagy részét kitöltő képek vannak magyar épületekről, szobrokról.

2. Elméleti háttér ismertetése

Ahhoz, hogy a fizikai térből származó vizuális információ feldolgozható és értelmezhető legyen a gépi látás számos részterületét érintő transzformációs láncot kell alkalmazni. Ez a fejezet azokat a matematikai és jelfeldolgozási elméleteket tárgyalja, amelyek a kártyafelismerő algoritmus gerincét alkotják.

2.1. Színterek konverziója és a digitális képreprezentáció

A folyamat legelején egy kétdimenziós diszkrét mátrixszal dolgozunk, amely a szenzor által rögzített fényintenzitás-értékeket tartalmazza. A szabványos színkódolás a 24 bites RGB színtér, ahol minden képpontot három, 0 és 255 közötti érték határoz meg. Az alakzatok azonosításához (élek, sarkok, kontúrok kereséséhez) azonban a színinformáció legtöbbször nemcsak felesleges, de jelentősen növeli az adattömeget és a számítási komplexitást.

Emiatt az első elméleti képfeldolgozási lépés a többcsatornás színes kép redukálása egyetlen csatornát tartalmazó szürkeárnyaltos képpé. Matematikailag ez a három csatorna súlyozott lineáris kombinációjával történik, az emberi szem eltérő frekvenciaérzékenységet (a zöld szín fokozott, a kék szín csökkent érzékelését) figyelembe véve.

Ennek eredménye egy kontrasztra és világosságkülönbségekre korlátozott térbeli mátrix, amellyel a geometriai és gradiens-alapú műveletek stabilan és determinisztikusan elvégezhetők.

2.2. Térbeli szűrés és Gaussi zajcsökkentés

A fizikai kamerák expozíciója során szinte mindig keletkezik képzaj. Mivel az éldetektálás a képpontok közötti különbségekre érzékeny, zajos képen minden apró képpont-ingadozást hamis élként azonosíthat. Ennek megelőzésére térbeli aluláteresztő szűrőket (low-pass filter) kell alkalmazni.

Erre ad megoldást a konvolúció, ami során egy kétdimenziós operátort (maszkot vagy kernelt) végig csúsztatunk a kép minden pontján, és a szűrő alatti képpontok intenzitását a szűrő súlyainak megfelelően átlagoljuk. A legelterjedtebb megoldás, ha a maszk súlyait a kétdimenziós, normál eloszlású Gauss-függvény alapján számítjuk ki.

2.3. Iránymenti gradiensek és éldetektálás

A korábbi lépések lezajlása után a kép olyan állapotba kerül, ami alkalmas a kártyalapok detektálására. A kép azon pontjait keressük, ahol a világosságértékek hirtelen és drasztikusan megváltoznak. Ezeket az átmeneteket intenzitásgradienseknek hívjuk. Ennek egyik leghatékonyabb eljárása a Canny-féle éldetektáló algoritmus, amely egy többlépcsős lánc.

2.3.1. Gradiens irányának és nagyságának meghatározása

Egy differenciáloperátorral (például a Sobel operátorral) meghatározzuk a képpontok változási sebességét horizontálisan és vertikálisan.

2.3.2. Nem-maximális elnyomás

A kapott élek általában viszonylag vastagok, olykor több pixel szélesek. Az algoritmus végigmegy minden talált élponton, és csak azokat tartja meg, amelyek a saját gradiens-irányuk mentén lokális maximumok. A többi pont értékét minimálisra állítja, ezáltal hajszálvékony, egy pixel szélességű élkép jön létre.

2.3.3. Hiszterézises küszöbölés

Két paramétert alkalmaz (egy alsó és felső küszöböt) az élek hitelesítéséhez. A magas küszöbérték feletti gradiensek "biztos" élek. A két küszöb közötti értékek csak akkor lesznek elfogadva élként, ha topológiai és folytonosan kapcsolódnak egy biztos élhez. Ezzel kiszűrjük a gyenge, szeparált zajos átmeneteket.

2.4. Matematikai morfológia alkalmazása

Bármilyen kiváló is egy éldetektáló algoritmus, előfordulhatnak az állandó élek mentén kisebb megszakadások (például egy árnyék vagy a kártyalap kopása miatt). Ezek megnehezítik az alakzatok zárt sokszögeként történő azonosítását. Ezen a ponton avatkozik be a matematikai morfológia; konkrétan a dilatáció (tágítás) módszere.

A dilatáció során egy strukturáló elemet (például egy 3 x 3 pixeles, csak egyeseket tartalmazó blokkot) söprünk végig az élképen. Ha a strukturáló elem területén bárhol található aktív (él) pixel, a középponti elem is aktívvá válik. Ez kvázi "megvastagítja" a Canny által kiszámolt finom vonalakat, összekötve a közeli megszakadt végpontokat, hogy később zárt görbeként legyenek vizsgálhatók. Egy másik morfológiai művelet a zárás (closing), ami egy dilatációt egy azt követő erózió (szűkítés) követ – ez előszeretettel alkalmazható kisebb, szimbólumok belsejében lévő hézagok eltüntetésére úgy, hogy a befoglaló alakzat területe ne növekedjen véglegesen.

2.5. Kontúrkeresés és poligon-approximáció

A zárt élvonalak megléte után következik a topológiai folytonosság elemzése, vagyis a kontúrkeresés. A számítógép a bináris képen szomszédsági algoritmusokkal végigköveti és csoportosítja az érintkező élpixelet. A kapott pontthalmazok alkotják a vizsgált alakzatokat, és képesek hordozni fahierarchiában lévő attribútumokat (például melyik kontúr melyik belsejében található). Amennyiben egy tárgyat keresünk, a legkülső, illetve kellően nagy területű körvonalakkal érdemes tovább dolgozni.

Mivel a fizikai testek felülete a képeken sosem szabályos matematikai görbék, hanem több száz "cikk-cakkozó" pixelből áll, ezek geometriai analízise pazarló és instabil. Ilyenkor a kontúrvonalakon sokszög-approximációt hajtunk végre. Az algoritmus elhagyja azokat a pontokat, amelyek elhanyagolható (egy limitált küszöbértéknél kisebb) távolságra térnek csak ki a sarkok által húzott virtuális egyenesektől. Ennek eredményeképpen egy négyszög alakú kártya megtalált körvonala pontosan négy sarokponttal írható le. Ha a kép szélei túl töredezetek a precíz approximációhoz, a forgatott minimalizált befoglaló téglalap (Minimum Area Bounding Rectangle) módszere is azonosíthatja a határoló sarokpontokat a teljes pontthalmaz konvex burka alapján.

2.6. Geometriai projektivitás és a Homográfia leképezés

Ha a kártya 4 sarka megvan, komoly geometriai torzítással is szembe kell néznünk. Mivel a kamerák a háromdimenziós, kiterjedt világot egy kétdimenziós szenzorra vetítik, de a tengelyük ritkán teljesen merőleges a felületre, látványos perspektivikus eltolódás következik be. Egy valójában téglalap alakú objektum a képen trapézként vagy torz négyszöggént jelenik meg.

Sík felületek esetén két látószög – azaz a rögzített ferde nézet és az idealizált felülnézet – geometriai viszonyát a projektív geometria segítségével, egy úgynevezett homográfia mátrixszal számíthatjuk ki. Ez egy 3×3 méretű mátrix, amivel végigszorozva a kép eredeti (x,y) sarokpontjainak koordinátáit, beléjük illesztve az új, virtuális origókat és kiterjedéseket (x',y') , felírhatók az egyenletrendszerek. A 8 szabadságfokkal rendelkező egyenlet a négy sarokpont megfeleltetésével egyértelműen kiszámítható. Ezután a kép belső képpontjaira bilineáris interpolációval leképezzük a cél pixelek világosságértékét. Az eredmény egy tökéletesen standard méretre és állásba formázott kép a torz kártyáról, aminek már az összes belső aránya torzításmentes számokra redukálódott.

2.7. Binarizálás a lokális területeken (Otsu-módszere)

A kártya méreteinek szabványosítása után csupán apró régiókkal (Jelen esetben a bal felső sarokkal) foglalkozva folytatjuk az alakzatkeresést. Ekkor már nem szürkeárnyalatos értékekre, hanem vegytiszta bináris maszkra van szükség a tényleges jelzések leírására.

Erre kínál megoldást az Otsu-algoritmus, ami végignézi az összes lehetséges értéket 0 és 255 között. Olyan küszöböt keres, ami minimalizálja az egymással szembenálló képosztályok, vagyis az előtérre leválasztott pixelek és a háttérként maradó pixelek belső szórását. Ez jelen esetben kifejezetten hatékony mert a kártyalapok esetében a háttér garantáltan fehér, a nekünk kellő rész pedig piros vagy fekete, így nem kell aggódni attól, hogy a háttér és az előtér összemosza.

2.8. Sablonillesztés és pontozó algoritmusok

A lánc legutolsó folyamata az izoláltan vizsgált szám, illetve szimbólum felismerése, amit statisztikai mintafelismeréssel hajtunk végre. Ehhez korábban megbízható binarizált sablonokat kell készíteni ugyanarról a kártya típusról, ideális környezetben lefotózva, és egyetlen fix pixelnézetben elmentve, amiket majd folyamatosan lekérdezésre ad a szoftver kiértékelő modulja.

2.8.1. Template matching

Az operátor a kisméretű vizsgált régiót rácsúsztatja a betöltött tárolókra. A leggyakrabban a Négyzetes hibaösszegek normalizált változatát számoljuk. Pixelenként végig vonja az eltéréseit és négyzetüket összegzi. Minél jobban konvergál nullához, annál közelebb áll egymáshoz az előtér és az új maszk. Második dimenziója ennek a Keresztkorreláció normalizált formulája, amely arra utal, hogy statisztikailag a fehér részhalmoz mekkora korrelációban követi a sötétet. Itt a legmagasabb normalitás jelzi a hasonlóságot.

2.8.2. IoU metrika (Intersection over Union)

A mesterséges intelligencia egyik alap értékmérője a halmazműveleti átfedés. Ha két bináris képet egyszerű halmazokként tekintjük, az összemetszését osztják el a két kép uniójával (Jaccard index). Hiába hasonló alakú a két alakzat, ha egy pici kör sokkal apróbb méretet fut, ott az unió jóval magasabb, amivel diszkvalifikálható az összetévesztés lehetősége egy nagyobbra rajzolt aszimmetrikus jelel is.

2.8.3. Kontúr és alakzat elemző metrika (Match Shape)

Míg az előzőek izolált pixelstruktúrákat tekintettek át, lehetőség van pusztán a kódolt szegélyeken hasonlóságot keresni. Vonalak hosszúsága mentén és rotációs invariánst rögzítő "Hu nyomatékokkal" a képek mérethez való viszonya teljesen elpárologtatható. Két nagyon pici négyszög a topológiában teljesen azonos egy extrán kiterjedt téglalappal szemben.

Végül a sablonillesztés befejezése és elméleti biztos pontja nem hagyatkozhat csupán egy algoritmikus konklúzióra. Súlyozott függvénybe vezetve azokat kikerekedik egy matematikai összevont pontszám, ahol mind a három vizsgált algoritmus megfelelő konstansteljesítménye szűrően határol fel az utolsó eredmény megkapásáig, amely kiemeli melyik is a vizsgált élethű lap.

3. A megvalósítás terve és kivitelezése

Ebben a fejezetben az előzőekben felvázolt képfeldolgozási elméletek gyakorlati megvalósítása, a program felépítése és a használt algoritmusok konkrét implementációja kerül tárgyalásra. A megoldás során a cél egy olyan kártyafelismerő logikai lánc (pipeline) létrehozása volt, amely képes dinamikusan reagálni a képeken jelentkező fizikai torzulásokra, fényviszonyokra és a lehető legnagyobb hatékonysággal tud működni.

Az alkalmazás két futtatható fájlból áll, a `single_card_recognition.py` felel a teljes logikáért, futtatásával egy fájl ellenőrzése végezhető el. A másik fájl a `batch_evaluate.py`, aminek célja, hogy az összes (104) fájlra végig futtassa a pipeline-t és összehasonlítsa a kapott eredmények egy excel fájlban (`Cards.xlsx`) előre megadott elvárt eredményekkel és számszerű, illetve százalékos értékelést adjon.

3.1. Felhasznált technológiák és a rendszer architektúrája

Az alkalmazás python alapon működik. Ennek oka, hogy kiterjedt adattudományi, matematikai és gépi látásbeli ökoszisztémával rendelkezik. A megvalósítás során három kritikus külső könyvtár (library) került bevonásra:

3.1.1. OpenCV (cv2)

Nyílt forráskódú könyvtár, ami kifejezetten gépi látás, képfeldolgozás és gépi tanulás problémák megoldására lett létrehozva.

3.1.2. NumPy

A képek a memóriában többdimenziós NumPy tömbökként (ndarray) reprezentálódnak. A NumPy használata elengedhetetlen volt a gyors mátrixtranszformációkhoz, sorrendezési algoritmusokhoz és a halmazműveletek végrehajtásához.

3.1.3. Matplotlib

Az algoritmus egyes lépéseiről tud vizuális visszajelzést adni. Elsősorban a fejlesztés során volt használva, hogy visszajelzést kapjak, hogy hol és milyen módon csúszik félre az algoritmus. A végső változatban is elérhető.

3.1.4. Pandas

Adatkezelő és elemző könyvtár, amely elsősorban strukturált, táblázatos adatok feldolgozására szolgál. A projektben az algoritmus által felismert kártyák naplózásában, a batch kiértékelések statisztikáinak tárolásában, valamint a lapszámlálás aktuális állásának és valószínűségeinek nyomon követésében van szerepe. Használata jelentősen megkönnyíti a teszteredmények aggregálását és exportálását.

3.2. A projekt könyvtárstruktúrája és globális paraméterei

Az algoritmus paramétereinek finomhangolása a kódban globális konstansokként történt, ezáltal a teljes képi transzformációs folyamat arányai könnyen skálázhatók. A kártya standard vágási méretét a `CARD_WIDTH = 200` és `CARD_HEIGHT = 300` px értékek határozzák meg, a sablonillesztés közös vászna pedig `MATCH_CANVAS = 64` px-re lett konfigurálva 6 px-es biztonsági margóval. Ezen döntések biztosítják az irreleváns képrészletek miatti adatvesztés és a megfelelő felbontás fenntartásának egyensúlyát.

Az alkalmazás könyvtár struktúra tekintetében két almappa található meg. Az `images` mappa tartalmazza a tesztadatbázist, a `templates` mappa pedig további almappákban a template matchinghez használt rank és suit mintaképeket.

3.3. Kártyalap izolálása és perspektivikus korrekciója

A felismerés első számú feltétele a kártyalap lokalizálása és madártávlati (top-down) nézetbe hozása, melyet a szoftverben a `find_card_quad` és a `warp_card` metódusok valósítanak meg.

3.3.1. Kép beolvasása és szürkeárnyaltos konverzió

A feldolgozási folyamat legelső lépéseként a program beolvassa a fizikai fájlt (`cv2.imread`), majd a memóriában reprezentált RGB színes képet azonnal szürkeárnyaltossá konvertálja a `cv2.cvtColor` függvény `cv2.COLOR_BGR2GRAY` paraméterének hívásával. Ennek célja, hogy a szoftver eldobja az élkeresést csak nehezítő színinformációkat, és tisztán a világosság-különbségekre fókuszálhasson.

3.3.2. Kép skálázása és zajszűrés

Ezt a szürkeárnyaltos képet a `find_card_quad` függvény fogadja, ahol elsőként egy adaptív átméretezés történik. Ha a kép túlságosan nagy felbontású (szélessége meghaladja a 800 pixelt), a rendszer egy arányos scale faktorral lekicsinyíti. Ez a kompromisszum kritikus a valós idejű futási sebesség elérése, illetve a zajcsökkentés miatt. A méretarányos képre ezt követően egy 5 x 5-ös maszkú Gaussian Blur kerül, mely eltünteti az asztal és a papír textúráját.

3.3.3. Kontúrok keresése

Az elmosott képre a Canny-féle éldetektáló algoritmus kerül futtatásra 50-es alsó és 150-es felső küszöbértékkel. Az így keletkezett egypixeles éleket a program rögtön egy tágító (`dilate`) morfológiai operátorral megvastagítja, kiküszöbölve a kártya határvonalainak fizikai megszakadásait. A zárt formákat az OpenCV `findContours` függvénye azonosítja külső kontúrkereséssel.

A megtalált alakzatok közül az irracionálisan kicsiket egy beépített vizsgáló szűri ki (minimum területnek a felbontás 5%-át kell kitennie). A valid kontúrokon a szoftver fokozatosan engedékeny iterációval Douglas-Peucker poligon-approximációt kísérel meg gyanított eps hibahatárok mellett, addig, míg pontosan négyszög alakzatot nem talál.

3.3.4. Tartalék vonalvezetés

Ha az approximáció kudarcot vall – például erős kitakarás vagy extrém alacsony kontraszt esetén –, a program egy "vészhelyzeti" ágra ugrik. Ebben az esetben a Canny detektort jóval érzékenyebb paraméterekkel (15 alsó, 50 felső küszöb) lefuttatja, és az asztalon található minden 50 pixel feletti területet összeborít egy nagy ponthalmazzá. Erre a halmazra ráilleszti a minimális befoglaló téglalap (Minimum Bounding Rect) képletét, amit biztosítékként 15%-kal nagyobbra skáláz. Ezzel a megoldással garantálható a kontúr kivágásának sikere szélsőséges esetekben is.

3.3.5. Sarkok rendezése és síkba fordítás

Az azonosított 4 sarokpont geometriai sorrendjét az `order_points` metódus matematikai vizsgálata rendezi sorba. Polaritási szögek számításával, a tömegközépponthoz viszonyított szög, és a távolságok analizálásával a metódus megállapítja a kártya rövidebb éleit. Ez alapján stabilan határozza meg a Bal-Felső, Jobb-Felső, Jobb-Alsó, Bal-Alsó sorrendet portré tájolásba kényszerítve a lapot. Ezen sorrend már alkalmas a `warp_card` metódus számára a perspektíva-mátrix (Homográfia) kiszámítására. A visszatérési érték a transzformált kimenet: egy 200 x 300 pixeles egyenes, letisztított, a torzulásoktól mentes virtuális kártyalap.

3.4. Dinamikus szimbólumkivágás az index sarokból

Miután a kártya stabil téglalapba került, a rang és a szín kinyerését az `extract_dynamic_symbols` függvény koordinálja. A program előre definiált statikus koordinátákat (ROI - Region of Interest: koordináta [0,0] -tol kiterjedés [29, 150] pixelig) használ, hogy lokalizálja a bal felső sarkot. Itt azonban nem statikus vágást, hanem dinamikus objektum-izolációt használ a hibás nyomtatások kompenzálására.

A levágott kis fényképen a kód megkísérli felderíteni a két diszkrét szimbólumot a belső kontúrjaik alapján. Ehhez először a sarok képét feldolgozza egy előfeldolgozó metódussal, majd minden talált, de hibás anomáliát (irracionálisan nagy kiterjedés vagy a széltől túlságosan eltolt régió) töröl maszkírozással. Az apró, szeparált pacák logikai összekapcsolására egy durva vertikális szerkezeti elemmel rendelkező morfológiai zárást (`MORPH_CLOSE`) hajt végre a rendszer. Ezt

követően megállapítja az alakzatokat magukba foglaló határoló dobozokat (bounding boxes), és a két legjobban elkülönülő dobozt koordinátáik alapján elválasztva definiálja, mint „Rang” (felső) és „Suit” (alsó) lekérdezési célkép. Amennyiben az elsődleges (bal felső) tartományban a szimbólumok izolálása sikertelen (például kitakarás, sérülés vagy túlexponáltság miatt), az algoritmus tartalmaz egy biztonsági tartalék (fallback) lépést is. Ekkor a kártya ellentétes, jobb alsó sarkából vágja ki a megfelelő régiót, amelyet 180 fokkal elforgat, majd újra megkísérli a kontúrkeresést. Ez néhány esetben megmenti a kártyafelismerést.

3.5. Szimbólumok binarizálása és Normalizálása

A program a beadott mátrixot egy apróbb, 3 x 3 pixeles iteratív Gauss-szűréssel zajtalanítja, ezt követően pedig megkapja az elsődleges binarizálást. Mivel itt már elzártan dolgozunk az asztal megvilágítási problémáitól egy homogén térben, az applikáció egy automata, globális Otsu-küszöbölést végez (cv2.THRESH_BINARY + cv2.THRESH_OTSU). Az Otsu kiszámolja a tökéletes intenzitás-középpontot, ami makulátlan minőségű, lyukak nélküli szimbólumtesteket eredményez.

Végül a binarizált maszkokat újra vágja a szoftver a legbelső pontos illeszkedési boxra, és felskálázza őket egy egyedi 64 x 64 pixeles közös „vászonra” interpolálással (INTER_AREA), precízen a vászon középpontjára igazítva. A kimeneten lévő sablon is és a vizsgálandó vizuális darabka is így teljesen invariáns lesz minden kezdeti elcsúszásra.

Ezen folyamatok hatására a bemeneti képek formátuma teljes mértékben megegyezik a mintának használt képek formátumával.

3.6. A sablonillesztés megvalósítása és összevont kiértékelése

A program magja a best template match logikai modulja, amely bejárja az összes memóriába transzponált és normalizált sablont (egy alkalommal a Rangokra, majd a Színekre), és egymás után kiolvassa azok matematikai hasonlóságát a kártya levágott maszkjához képest.

Az alkalmazás három algoritmust használ a pontosság növelése érdekében és a kapott eredmények súlyozott átlagából határozza meg a végső eredményt (CCOEFF_NORMED [45%], CCORR_NORMED [35%] és a megfordított SQDIFF_NORMED szorzó [20%]). Ennek logikája megakadályozza a kontrasztok kiugrásaiból adódó lokális tévedéseket. A megírt `template_ensemble_score` hozza létre ezen fázis pontját.

3.7. Összekapcsolás és a rendszer végrehajtási (fő) folyamata

A kifejtett moduláris függvényeket a `recognize_single_card` fogja össze és indítja el transzlációsán egy futási szekvencia keretén belül. A folyamat beolvassa a

felhasználó által terminálon megadott (Argparse-szal feldolgozott image_path) külső képet.

A kód sorban meghívja a lokátor metódust, majd átfuttatja a vágásokon és normalizálásokon. Miután letárolta a vizsgált két szériát (kivágott rang, kivágott suit), egy glob iterátor használatával a memóriába konvertálja a konfigurációban megadott sablonkönyvtárak tartalmát, s belép az értékelő szakaszba. A visszkapott osztályozási címkéket adja ki a konzolos eredményre.

A program beépített egy vizuális debugolási (nyomkövető) módozatot is --debug paraméterezéssel. Aktiválása esetén a recognize_single_card a matematikai műveletek mögött egy a Matplotlib alrendszerbe épített subplot ablakon jeleníti meg ember által is fogyasztható módon az eredeti vizsgált fájlt, valamint az illesztésbe belekerülő torzulásmentes, kiizolált szimbólumokat, mellettük feltüntetve az algoritmus által dedikált kitalált döntéseket. Ez óriási mértékben támogatja a jövőbeli tesztelési modulok elkészítését és a hibák valós idejű feltárását azonosítását.

3.8. Tömeges kiértékelés és tesztelés

A kártyafelismerő algoritmus robusztusságának és pontosságának mérésére a rendszer egy automatizált köteget (batch) tesztelő modult is tartalmaz. Ennek feladata, hogy egy előre definiált adathalmazon iteráljon végig, a predikciókat összevesse a valós (ground truth) címkékkel, és statisztikát készítsen a felismerés hatékonyságáról. Ezt a folyamatot a batch_evaluate.py szkript hajtja végre.

3.8.1. A tesztadathalmaz és a bemenetek kezelése

A modul parancssori argumentumokon (argparse) keresztül veszi át az indítási paramétereket:

- Az Excel formátumú igazság-tábla fájl elérési útja (--excel)
- A bemeneti képeket tartalmazó könyvtár (--images-dir)
- A sablonok (színek és értékek) elérési útjai

A validációs adatok beolvasását a pandas könyvtár végzi. Az Excel fájl struktúrája egyszerű: az első oszlop a fájl nevét, a második a várt kártyaszínt (suit), a harmadik pedig a várt kártyaértéket (rank) tartalmazza. A rendszer a beolvasás során automatikusan eltávolítja a felesleges szököket, és ha szükséges, kiterjesztést (.jpg) illeszt a fájlnevek végére.

3.8.2. A tesztelési iteráció

A szkript egy for-ciklus segítségével halad végig az igazság-tábla sorain. Minden egyes bejegyzésnél megbizonyosodik róla, hogy a hivatkozott képfájl ténylegesen létezik a fájlrendszerben.

Ha a kép elérhető, a fájl meghívja a `single_card_recognition.py` modul fő belépési pontját, a `recognize_single_card` függvényt. Ez a függvény végrehajtja a teljes – a korábbi fejezetekben részletezett – feldolgozási láncot (kép beolvasása, előfeldolgozás, kontúrkeresés, perspektív transzformáció, ROI kinyerése, küszöbölés, és Végül a sablonillesztés), majd visszatér egy jóslat érték-szín (`pred_rank`, `pred_suit`) párossal, mindezt a vizualizációs lépések (`debug=False`) kihagyásával a gyorsabb lefutás érdekében.

3.8.3. Eredmények naplózása és összesítés

A predikciók megérkezése után a modul összeveti azokat a táblázatban szereplő elvárt értékekkel. Külön regisztrálja a kártyaérték (`rank`) és a kártyaszín (`suit`) egyezését. Naplózza a szabványos kimenetre az aktuális fájl eredményét. Hibás találat esetén pontosan kiírja a várt és az algoritmus által kapott eredményt, hogy megkönnyítse a tévesztések vizuális utóellenőrzését.

A teljes adathalmaz feldolgozása után a szkript kiszámítja a felismerési pontosságot mindkét tulajdonságra külön-külön, illetve a teljes egyezésre (ahol a szín és az érték is egyidejűleg helyes). A végeredmény egy konzolos összegzés, amely százalékos arányban fejezi ki az algoritmus validációs adathalmazon nyújtott teljesítményét.

4. Tesztelés

A teszteléshez két kártyapakli, tehát 104 lap volt használva. A fejlesztés nagy része alatt a kártyák egyesével voltak tesztelve, hogy a matplotlib könyvtár használatával vizuális visszajelzést kapjak, hogy milyen hibák merülnek fel az egyes lapoknál. Miután az alkalmazás elért egy elfogadható szintet és egy átfogó képet kaptam arról, hogy a kód mely részeit kell még fejleszteni, létrehoztam a `batch_evaluate.py` fájlt, aminek segítségével gyorsan tudtam pontos statisztikát kapni az alkalmazás hatékonyságáról.

4.1. Egy kártya tesztelése

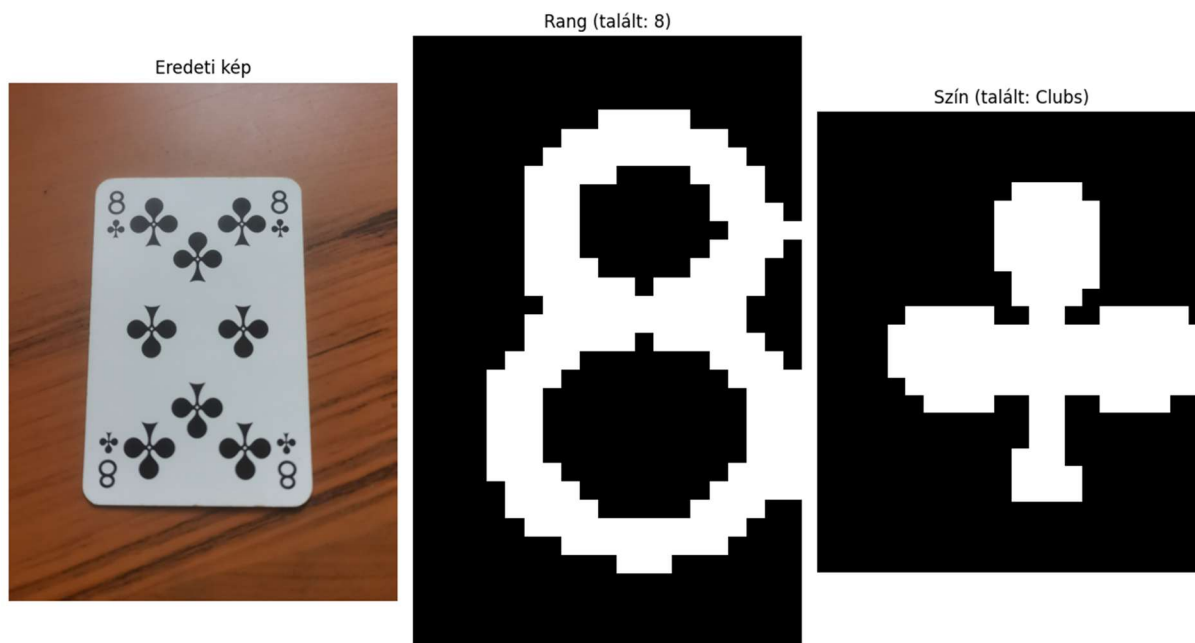
Ebben az esetben az alkalmazás csak a kiválasztott fájlon megy végig az alkalmazás.

Lehetséges kimenetek:

- Siker (ettől még nem garantált a helyes megoldás): Nincs kimenet a terminálon, megjelenik egy matplotlib GUI ablak, ahol látható az eredeti kép, a kivágott és binarizált rank és suit és a hozzájuk kapott eredmények.
- Hibaüzenetek:
 - A megadott képfájl nem létezik, vagy nem olvasható képformátum
RuntimeError: Nem sikerült beolvasni a képet: images\200.jpg
 - A Canny-élkereső és a kontúrkereső nem talált az előre megadott minimum területhez elégséges formát az eredeti képen
RuntimeError: Nem talált kártya-kontúrt
 - Talált kontúrt, de az alacsonyabb küszöbös Canny sem tudott téglalapot csinálni belőle
RuntimeError: Nem talált sem élt, sem belső objektumokat a kártyán!
 - Miután a kártya ki lett vágva, sem a bal felső ROI-ban ([0:150, 0:29]), sem (forgatás után) az ellenkező sarokban nem sikerült 2 darab ésszerű méretű szimbólum kontúrt binarizálni.
RuntimeError: Nem talált megfelelő szimbólumokat a sarokban

```
PS C:\Egyetem\VI. félév\Képfeldolgozás\kartya-szamlalo-alkalmazas> python single_card_recognition.py --image .\images\200.jpg --debug
[ WARN:0@0.017] global loadsave.cpp:278 cv::findDecoder imread_('images\200.jpg'): can't open/read file: check file path/integrity
Traceback (most recent call last):
  File "C:\Egyetem\VI. félév\Képfeldolgozás\kartya-szamlalo-alkalmazas\single_card_recognition.py", line 379, in <module>
    main()
    ~~~~^
  File "C:\Egyetem\VI. félév\Képfeldolgozás\kartya-szamlalo-alkalmazas\single_card_recognition.py", line 370, in main
    recognize_single_card(
    ~~~~~^
    image_path=args.image,
    ~~~~~^
    ...<2 lines>...
    debug=args.debug
    ~~~~~^
    )
    ^
  File "C:\Egyetem\VI. félév\Képfeldolgozás\kartya-szamlalo-alkalmazas\single_card_recognition.py", line 311, in recognize_single_card
    raise RuntimeError(f"Nem sikerült beolvasni a képet: {image_path}")
RuntimeError: Nem sikerült beolvasni a képet: images\200.jpg
```

1.kép – Példa sikertelen képbeolvasásra



2. kép – Példa helyes kártyafelismerésre

```
PS C:\Egyetem\VI. félév\Képfeldolgozás\kartya-szamlalo-alkalmazas> py single_card_recognition.py --image .\images\teszt.jpg --debug
Traceback (most recent call last):
  File "C:\Egyetem\VI. félév\Képfeldolgozás\kartya-szamlalo-alkalmazas\single_card_recognition.py", line 379, in <module>
    main()
  File "C:\Egyetem\VI. félév\Képfeldolgozás\kartya-szamlalo-alkalmazas\single_card_recognition.py", line 370, in main
    recognize_single_card(
    ~~~~~^
        image_path=args.image,
        ~~~~~^
    ...<2 lines>...
        debug=args.debug
        ~~~~~^
    )
    ^
  File "C:\Egyetem\VI. félév\Képfeldolgozás\kartya-szamlalo-alkalmazas\single_card_recognition.py", line 314, in recognize_single_card
    quad = find_card_quad(image_gray)
  File "C:\Egyetem\VI. félév\Képfeldolgozás\kartya-szamlalo-alkalmazas\single_card_recognition.py", line 78, in find_card_quad
    raise RuntimeError("Nem talált kártya-kontúrt")
RuntimeError: Nem talált kártya-kontúrt
```

3. kép – Példa olyan esetre, ahol a program nem talált kontúrt a kártyán

4.2. Összes kártya tesztelése

Ebben az esetben az alkalmazás végigmegy az images mappa összes fájlján és összehasonlítja a kapott eredményeket egy excel táblában előre meghatározott várt eredményekkel. Terminálon visszajelzést ad minden kártya eredményéről egyesével, majd a végén összegzi a kapott eredményeket.

Lehetséges kimenetek:

- Siker pl.: OK -> 69.jpg: Jack_Clubs
- A kapott eredmény és nem egyezik az excel táblában lévő helyes megoldással

pl.: TÉVESZTÉS -> 28.jpg | Rank: OK | Suit: HIBA (várt: Spades, kapott: Clubs)

- Hiba a kép feldolgozása során

pl.: HIBA FELDOLGOZÁSKOR -> teszt.jpg: Nem talált kártya-kontúrt

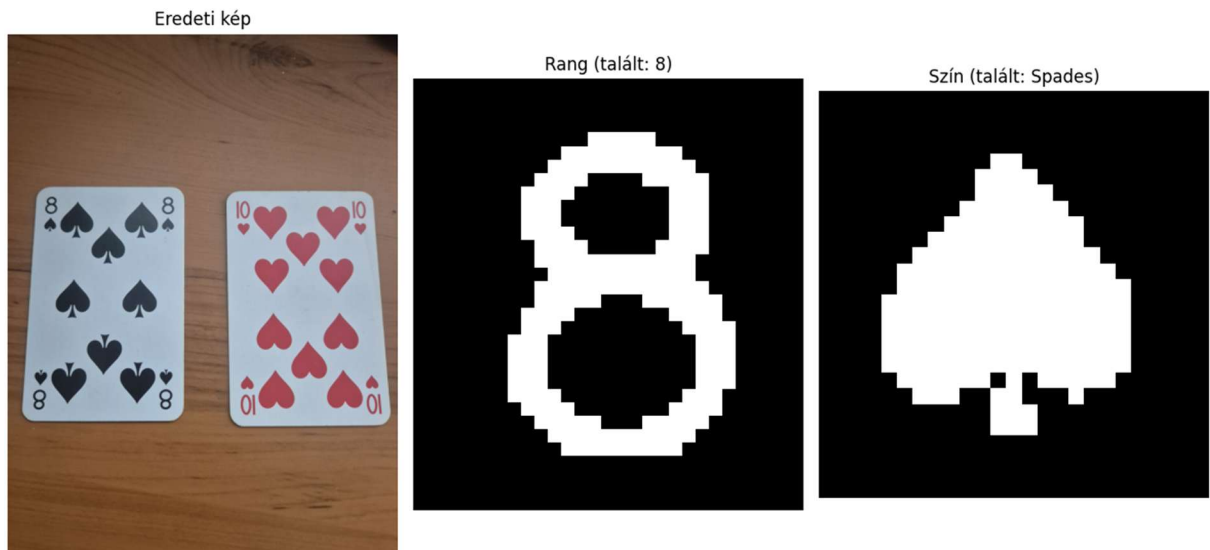
Összesítés:

```
=====
ÖSSZESÍTÉS (104 feldolgozott elem alapján):
Rank helyes   : 97 / 104 -> 93.27%
Suit helyes   : 100 / 104 -> 96.15%
Teljes egyezés: 96 / 104 -> 92.31%
=====
```

4.kép – Összesített eredmények

4.3. Több kártya a képen

Az alkalmazás alapvetően egy kártya felismerésére van felkészítve, viszont kipróbáltam mi van akkor, ha két kártya van egy képen. Ebben az esetben az alkalmazás meg tudja találni a kamerához kicsit közelebb lévő kártyát és azzal végig tud menni a folyamatokon.



5.kép – Példa arra az esetre, amikor két kártya van egy képen

4.4. Jellemző hibaforrások és tévesztések

4.4.1. Suit-ok hasonlósága

A globális küszöbölés következtében a képek elvesztik az eredeti színinformációjukat. Emiatt a sablonillesztés kizárólag a topológiai hasonlóságra hagyatkozhat. Emberi látással jól elkülöníthetők a szimbólumok, mert ismerjük sajátosságait. A template matching viszont nem ismeri ezeket a sajátosságokat, így főleg abban az esetben, ha a megvilágítás vagy kameraszög miatt nem pontos a bemenet könnyen tud téveszteni.

4.4.2. Értékek felcserélése

Hasonló karakterisztikájú számok és betűk esetén az algoritmus hajlamos azok felcserélésére (pl.: 6 és 9 vagy 3 és 8).

4.4.3. Kártya elcsúszása

Amennyiben a kártyalap torzítva lett normalizálva, a betűk és szimbólumok túllóghatnak a vágási területen, vagy éppen a kártya közepéről lóghatnak be szimbólumok be oda. Emiatt a program hajlamos levágni a szimbólumok széleit vagy éppen belevenni olyan kontúrokat, amiket nem kéne és ez teljesen félre tudja venni a template matchinget. Ennek megoldására próbálgatással próbáltam megtalálni az aranyközéputat, ahol akkora a nézett sarok mérete, ahol ez a lehető legkevesebbszer okoz problémát.

4.4.4. Eltérő háttér

A tesztadatbázisként használt képek nagy része ugyan azzal a háttérrel készült, azonban próbáltam más háttérrel készült képeket is. Ezekben az esetekben elég pontatlanul teljesített a program.

5. Felhasználói leírás

Az alkalmazás futtatásához Python környezet (ajánlott verzió: 3.8 vagy újabb) szükséges. A felhasznált külső könyvtárakat (OpenCV, NumPy, Pandas, Matplotlib, openpyxl) a projektben található requirements.txt fájl segítségével lehet telepíteni a parancssorból egyetlen paranccsal:

```
pip install -r requirements.txt
```

5.1. Projekt könyvtárszerkezete

single_card_recognition.py: Fő kártyafelismerő modul

batch_evaluate.py: Tömeges kiértékelést végző modul

requirements.txt: Függőségek listája

Cards.xlsx: Tömeges kiértékeléshez használt excel tábla, ami segítségével ellenőrzi az eredmény helyességét

images: Teszteléshez, felismeréshez szánt bemeneti képek (.jpg formátum)

templates:

rank: A kártyaértékek (A, 2-10, J, Q, K) binarizált sablonképeit tartalmazza.

suit: A kártyaszínek (káró, kör, pikk, treff) binarizált sablonképeit tartalmazza.

Új képek felvételéhez az images mappába kell feltölteni a képeket .jpg formátumban. Ahhoz, hogy a kép a tömeges kiértékelésben is részt vegyen, ahhoz a Cards excel táblázatba is fel kell venni. Név (kiterjesztés nélkül), suit, rank formátumban.

5.2. Egyetlen kártya felismerése

A modul egyetlen kép adatait képes feldolgozni, elvégezve a teljes vágási, küszöbölési és sablonillesztési folyamatot. Indítása a parancssorból történik.

```
python single_card_recognition.py --image "images/teszt_kartya.jpg" --debug
```

Futtatás után egy matplotlib GUI nyílik meg, ahol látszódik a bemenet és a kapott eredmények.

5.3. Tömeges tesztelés és kiértékelés

Nagyobb adathalmazok ellenőrzéséhez és az algoritmus pontosságának méréséhez a `batch_evaluate.py` szkript nyújt segítséget.

Futtatása:

```
python batch_evaluate.py --excel "Cards.xlsx"
```

A futtatás végén a rendszer a konzolra kiírja, hogy az adathalmaz hány százalékában volt sikeres a felismerés, kiemelve a kártyaszínre és kártyaértékre vonatkozó részeredményeket, illetve a konkrét tévesztéseket a könnyebb visszakövethetőség érdekében.

6. Irodalomjegyzék

- [1] „OpenCV: Template Matching”. Elérés: 2026. május 17. [Online]. Elérhető: https://docs.opencv.org/4.x/d4/dc6/tutorial_py_template_matching.html

- [2] „OpenCV: Contours in OpenCV”. Elérés: 2026. május 17. [Online]. Elérhető: https://docs.opencv.org/4.x/d3/d05/tutorial_py_table_of_contents_contours.html

- [3] „OpenCV: Canny Edge Detection”. Elérés: 2026. május 17. [Online]. Elérhető: https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

- [4] A. Rosebrock, „4 Point OpenCV getPerspective Transform Example”, PyImageSearch. Elérés: 2026. május 17. [Online]. Elérhető: <https://pyimagesearch.com/2014/08/25/4-point-opencv-getperspective-transform-example/>

- [5] D. Shah, *dharm1k987/Card_Recognizer*. (2026. április 24.). Python. Elérés: 2026. május 17. [Online]. Elérhető: https://github.com/dharm1k987/Card_Recognizer

- [6] Evan, *EdjeElectronics/OpenCV-Playing-Card-Detector*. (2026. április 28.). Python. Elérés: 2026. május 17. [Online]. Elérhető: <https://github.com/EdjeElectronics/OpenCV-Playing-Card-Detector>